

CE802 (2020) Worked Solutions

Machine Learning and Data Mining — full worked answers with formulas, calculations, and brief interpretation.

Based on the uploaded 2020 CE802 exam paper. The aim here is not to jump straight to the final option/decision, but to show the reasoning that leads to it.

Question 1 — Reinforcement Learning / MDP

(a) Modelling the sailor game as an MDP

A convenient state description is the town where the sailor is at the start of the day, plus one terminal failure state for a sunk boat.

State space:

$S = \{A, B, X\}$, where X is an absorbing terminal state ("boat sunk / game over").

At towns A and B the sailor can either stay put or attempt one crossing with 1, 2, or 3 passengers. The terminal state X has no useful actions.

Action sets:

$A(A) = A(B) = \{\text{stay, cross}_1, \text{cross}_2, \text{cross}_3\}$; $A(X) = \emptyset$.

Current state s	Action a	Non-zero transition probabilities $T(s' s,a)$	Immediate reward $R(s,a)$
A	stay	$T(A A,\text{stay})=1$	-10
B	stay	$T(B B,\text{stay})=1$	-10
A	cross_n ($n \in \{1,2,3\}$)	$T(B A,\text{cross}_n)=1-n/10$; $T(X A,\text{cross}_n)=n/10$	$50n - 10$
B	cross_n ($n \in \{1,2,3\}$)	$T(A B,\text{cross}_n)=1-n/10$; $T(X B,\text{cross}_n)=n/10$	$50n - 10$

Why these rewards are correct: the fare is collected before boarding, so carrying n passengers gives $+50n$ even if the boat later sinks; every day costs 10 points regardless of the action, so we subtract 10 in every non-terminal state.

(b) Meaning of Bellman's optimality equation

For deterministic transitions,

$$V^*(s) = \max_{a \in A(s)} [R(s,a) + \gamma V^*(s')]]$$

means that the optimal value of a state s is the best discounted return achievable from s . For each available action a , we add:

- the immediate reward $R(s,a)$, and
- the discounted optimal value $\gamma V^*(s')$ of the unique successor state s' reached after taking a .

The max operator chooses the action that gives the largest total. So the equation is a recursive "best future return" definition.

(c) Discounted cumulative values for the graph in Figure 1.1, with $\gamma = 0.6$

From the state-transition diagram on page 2 of the exam paper, the available deterministic moves are:

State	Available next states	Reward on transition
A	C	0
B	C or E	0
C	A, D or E	0

D	A or G1	60 on D→G1; 0 on D→A
E	C or G2	70 on E→G2; 0 on E→C
G2	G1 or G3	60 on G2→G1; 90 on G2→G3
G1	terminal	0
G3	terminal	0

We work backward from terminal states. Since G1 and G3 have no outgoing rewarding actions, we take $V(G1)=0$ and $V(G3)=0$.

State	Bellman calculation	Value
G2	$V(G2)=\max(60+0.6 \cdot V(G1), 90+0.6 \cdot V(G3)) = \max(60, 90)$	90
E	$V(E)=\max(0+0.6 \cdot V(C), 70+0.6 \cdot V(G2)) = \max(0.6V(C), 70+54)$	124
C	$V(C)=\max(0.6V(A), 0.6V(D), 0.6V(E)) = \max(0.6V(A), 0.6V(D), 74.4)$	74.4
A	$V(A)=0+0.6 \cdot V(C)=0.6 \cdot 74.4$	44.64
D	$V(D)=\max(0+0.6 \cdot V(A), 60+0.6 \cdot V(G1)) = \max(26.784, 60)$	60
B	$V(B)=\max(0.6V(C), 0.6V(E)) = \max(44.64, 74.4)$	74.4

So the discounted cumulative state values are:

$V(A)=44.64$, $V(B)=74.4$, $V(C)=74.4$, $V(D)=60$, $V(E)=124$, $V(G1)=0$, $V(G2)=90$, $V(G3)=0$.

(d) Role of the discount factor γ

The discount factor controls how much future rewards matter compared with immediate rewards.

- Small γ : the agent is short-sighted and prefers earlier rewards.
- Large γ : the agent is far-sighted and is willing to wait for larger later rewards.

At state D there is a clear trade-off:

- go directly to G1 and receive 60 immediately; or
- go to A, then optimally $A \rightarrow C \rightarrow E \rightarrow G2 \rightarrow G3$, collecting 70 and 90 later.

The discounted value of choosing $D \rightarrow A$ and then following the best route is

$$0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 70 + \gamma^4 \cdot 90 = 70\gamma^3 + 90\gamma^4.$$

The direct route $D \rightarrow G1$ is worth 60. Therefore the policy at D changes when

$$70\gamma^3 + 90\gamma^4 = 60.$$

Solving numerically gives $\gamma \approx 0.7573$. Hence:

- if $\gamma < 0.7573$, the optimal action at D is to go straight to G1;
- if $\gamma > 0.7573$, the delayed but larger reward path via A becomes optimal;
- at $\gamma = 0.7573$, the two choices tie.

For the exam value $\gamma = 0.6$, the optimal move from D is therefore $D \rightarrow G1$.

Question 2 — Naïve Bayes

(a) The two probability equations

(i) $P(A|B) = P(A, B) / P(B)$ is the definition of conditional probability. It tells us the probability of A restricted to cases where B has occurred.

(ii) $P(A|B) = P(B|A)P(A) / P(B)$ is Bayes' theorem. In machine learning it lets us reverse a probability that is easy to estimate (likelihood times prior) into the posterior probability of a class given observed features.

(b) Designing the Naïve Bayes classifier from the 7 training examples

The class variable is Upgrade $\in \{\text{Approved, Rejected}\}$. Count the class frequencies first.

Class	Count	Prior probability	Decimal
Rejected	4	$P(\text{Rejected})=4/7$	0.5714
Approved	3	$P(\text{Approved})=3/7$	0.4286

Next estimate all conditional probabilities by relative frequency within each class.

Feature/value	Within Rejected (4 cases)	Probability	Within Approved (3 cases)	Probability
Credit=Good	2	$2/4 = 1/2$	2	$2/3$
Credit=Poor	2	$2/4 = 1/2$	1	$1/3$
Income=Low	3	$3/4$	1	$1/3$
Income=High	1	$1/4$	2	$2/3$
Card=Yes	1	$1/4$	1	$1/3$
Card=No	3	$3/4$	2	$2/3$

The Maximum-A-Posteriori (MAP) decision rule is therefore:

Choose the class c that maximises $P(c) \cdot P(\text{Credit}=x_1|c) \cdot P(\text{Income}=x_2|c) \cdot P(\text{Card}=x_3|c)$.

(c) Classifying a customer with Good credit, Low income, and more than one card

Compute the unnormalised posterior score for each class.

Class	Calculation	Score
Rejected	$(4/7) \cdot (1/2) \cdot (3/4) \cdot (1/4)$	$3/56 \approx 0.05357$
Approved	$(3/7) \cdot (2/3) \cdot (1/3) \cdot (1/3)$	$2/63 \approx 0.03175$

Because $3/56 > 2/63$, the classifier assigns the application to the Rejected class.

(d) Why a shallow decision tree with a Naïve Bayes model at each leaf can outperform both DT and NB

The proposed cascade combines complementary strengths.

- Compared with a standard decision tree: a shallow tree avoids over-fragmenting the data. Instead of using only a majority label at a leaf, the leaf-level Naïve Bayes model still uses the remaining features, so more information is exploited and variance can be reduced.
- Compared with a standard Naïve Bayes model: the top tree splits on a few especially informative features, creating local regions where the remaining features are more nearly conditionally independent. This weakens the global-independence assumption that often hurts standard NB.
- The tree captures important interactions or regime changes; the NB classifier inside each region gives a compact probabilistic model instead of a very deep, brittle set of splits.

So the hybrid can be better because it keeps the tree simple, while allowing each leaf to make a richer probabilistic decision than a pure decision tree.

Question 3 — Association Rule Mining / Apriori

(a) Meaning of the probabilistic definitions for a rule $J \Rightarrow K$

- (i) $P(K|J)$: confidence. Among transactions containing J , it is the proportion that also contain K .
- (ii) $P(K \cup J)$: support. It is the proportion of transactions containing all items in J and K together.

(iii) $P(K|J) / P(K)$: lift. It measures how much more likely K is when J is present than in the database overall. Lift > 1 indicates positive association, lift = 1 independence, lift < 1 negative association.

(b) Apriori with minimum support 25%

There are 10 transactions, so the minimum support count is $0.25 \times 10 = 2.5$, hence an itemset must appear at least 3 times to be frequent.

Level 1: single items

Candidate	Support count	Frequent?	Reason if discarded
{A}	7	Yes	
{B}	7	Yes	
{C}	8	Yes	
{D}	2	No	insufficient support
{E}	3	Yes	
{F}	1	No	insufficient support
{G}	1	No	insufficient support
{H}	1	No	insufficient support

Therefore $L1 = \{\{A\}, \{B\}, \{C\}, \{E\}\}$.

Level 2: generate pairs from L1

Candidate	Support count	Frequent?	Reason if discarded
{A,B}	5	Yes	
{A,C}	5	Yes	
{A,E}	3	Yes	
{B,C}	5	Yes	
{B,E}	2	No	insufficient support
{C,E}	3	Yes	

Therefore $L2 = \{\{A,B\}, \{A,C\}, \{A,E\}, \{B,C\}, \{C,E\}\}$.

Level 3: join and prune using Apriori

Candidate	All 2-subsets frequent?	Support count	Frequent?	Reason if discarded
{A,B,C}	Yes	3	Yes	
{A,B,E}	No (subset {B,E} not frequent)	-	No	discarded by Apriori property
{A,C,E}	Yes	3	Yes	
{B,C,E}	No (subset {B,E} not frequent)	-	No	discarded by Apriori property

Therefore $L3 = \{\{A,B,C\}, \{A,C,E\}\}$.

Level 4:

The only join candidate is {A,B,C,E}, but it is pruned immediately because two of its 3-subsets, {A,B,E} and {B,C,E}, are not frequent. So there is no need to count its support.

Hence the full set of frequent itemsets is:

$\{A\}, \{B\}, \{C\}, \{E\}, \{A,B\}, \{A,C\}, \{A,E\}, \{B,C\}, \{C,E\}, \{A,B,C\}, \{A,C,E\}$.

(c) Strong rules of the form $\{X,Y\} \Rightarrow \{Z\}$ with confidence threshold 75%

Only frequent 3-itemsets can generate rules with two items on the left and one on the right. We test all such rules.

Rule	Confidence formula	Confidence	Strong?	Lift
$\{A,B\} \Rightarrow \{C\}$	$\frac{\text{sup}(ABC)}{\text{sup}(AB)} = 3/5$	0.60	No	$0.60 / 0.80 = 0.75$
$\{A,C\} \Rightarrow \{B\}$	$\frac{\text{sup}(ABC)}{\text{sup}(AC)}$	0.60	No	$0.60 / 0.70 \approx 0.857$

	$\text{sup}(AC)=3/5$			
$\{B,C\} \Rightarrow \{A\}$	$\text{sup}(ABC)/\text{sup}(BC)=3/5$	0.60	No	$0.60 / 0.70 \approx 0.857$
$\{A,C\} \Rightarrow \{E\}$	$\text{sup}(ACE)/\text{sup}(AC)=3/5$	0.60	No	$0.60 / 0.30 = 2.00$
$\{A,E\} \Rightarrow \{C\}$	$\text{sup}(ACE)/\text{sup}(AE)=3/3$	1.00	Yes	$1.00 / 0.80 = 1.25$
$\{C,E\} \Rightarrow \{A\}$	$\text{sup}(ACE)/\text{sup}(CE)=3/3$	1.00	Yes	$1.00 / 0.70 \approx 1.429$

Therefore the strong rules of the required form are $\{A,E\} \Rightarrow \{C\}$ and $\{C,E\} \Rightarrow \{A\}$.

(d) Why Apriori is cheaper than brute force

The database contains 8 distinct items (A–H). A brute-force search would in principle need to evaluate support for all non-empty itemsets:

$$2^8 - 1 = 255 \text{ itemsets.}$$

Apriori is much cheaper because it uses the downward-closure property: if an itemset is infrequent, then every superset of it is automatically infrequent. This enables pruning before support counting.

In this exercise, support actually had to be counted only for:

- 8 singleton candidates;
- 6 two-item candidates generated from L1;
- 2 three-item candidates that survived subset pruning ($\{A,B,C\}$ and $\{A,C,E\}$).

So only $8 + 6 + 2 = 16$ support computations were needed, compared with 255 for brute force. The saving comes from the level-wise candidate generation plus immediate pruning of any candidate whose subset is already known to be infrequent.

End of worked solutions

Each answer above was derived step by step from the uploaded 2020 CE802 paper rather than presented as an unexplained answer key.